Storm-Gauge Response Model — Test Results

Automated Test Documentation for Model Governance

MKM Research Labs

Version 1.0 — 26-March-2026

David K Kelly

Legal Notice

PROPRIETARY AND CONFIDENTIAL

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs ("MKM").

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an "as is" basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

1 Test Results — Storm-Gauge Response Model (MKM-SG-001)

Tests run on **26 March 2026 at 18:58:34**.

Table 1: Test Results Summary — Storm-Gauge Response Model (MKM-SG-001)

Total Tests	81
Passed	81
Failed	0
Skipped	0
Duration	0.03s

Table 2: Detailed Test Results — Storm-Gauge Response Model

Test Description	Acceptance Criteria	Result	Time
Gauge ids match	`{r.gauge_id for r in responses} == {gauge.gauge_id, gauge...`	Pass	0.000s
Near gauge higher response	`near_resp.peak_level >= far_resp.peak_level`	Pass	0.000s
Returns one response per gauge	`len(responses) == 2`	Pass	0.000s
All kernels decrease with distance	`near >= far`	Pass	0.000s
All kernels in 0 1 range	`0.0 <= decay <= 1.0`	Pass	0.000s
Exponential at half	`decay == pytest.approx(math.exp(-0.5), rel=1e-4)`	Pass	0.000s
Exponential at one lambda	`decay == pytest.approx(math.exp(-1.0), rel=1e-4)`	Pass	0.000s
Gaussian at one sigma	`decay == pytest.approx(math.exp(-0.5), rel=1e-4)`	Pass	0.000s
Gaussian at two sigma	`decay == pytest.approx(math.exp(-2.0), rel=1e-4)`	Pass	0.000s
Linear beyond cutoff is zero	`decay == 0.0`	Pass	0.000s
Linear midpoint	`decay == pytest.approx(0.75, abs=0.001)`	Pass	0.000s
Zero distance returns one all kernels	`model.compute_decay(0, 100, kernel, 1.0) == 1.0`	Pass	0.000s
After end uses last point	`intensity >= 0`	Pass	0.000s
At peak time is nonzero	`intensity > 0`	Pass	0.000s
Before start uses first point	`intensity >= 0`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Empty track returns zero	`result == 0.0`	Pass	0.000s
Far from track is lower	`near > far`	Pass	0.000s
Accumulation nonnegative	`resp.accumulation >= 0`	Pass	0.000s
Duration nonnegative	`resp.duration_above_alert >= 0`; `resp.duration_above_warning >= 0` (+1 more)	Pass	0.000s
Flooded flag correct	`resp.flooded == (resp.peak_level >= gauge.flood_alert)`	Pass	0.000s
A storm passing directly over with peak=100 should flood the gauge.	`resp.peak_level > gauge.base_level`	Pass	0.000s
Peak level at least base	`resp.peak_level >= gauge.base_level`	Pass	0.000s
Peak level is max of series	`resp.peak_level == pytest.approx(max(levels))`	Pass	0.000s
Response ids match	`resp.gauge_id == gauge.gauge_id`; `resp.storm_id == simple_storm.storm_id`	Pass	0.000s
Timeseries length	`len(resp.level_timeseries) == expected_steps`; `len(resp.intensity_timeseries) == expected_steps`	Pass	0.000s
To dict roundtrip	`d['gauge_id'] == gauge.gauge_id`; `'level_timeseries' in d` (+1 more)	Pass	0.000s
To dict without timeseries	`'level_timeseries' not in d`	Pass	0.000s
Encodes datetime	`"2026-01-01" in result`	Pass	0.000s
Line 80: np.float32 is np.floating but NOT a Python float subclass in NumPy 2.x,	`"2.5" in result`	Pass	0.000s
Encodes numpy floating	`"3.14" in result`	Pass	0.000s
Encodes numpy integer	`"42" in result`	Pass	0.000s
Encodes numpy ndarray	`"1.0" in result`	Pass	0.000s
Fallback raises for unknown	—	Pass	0.000s
Distance is positive	`dist > 0`	Pass	0.000s
Returns nearest of three	`nearest.intensity == 80.0`; `dist == pytest.approx(0.0, abs=1.0)`	Pass	0.000s
Single track point	`nearest is track[0]`	Pass	0.000s
Close to thames high risk	`result["FloodRiskScore"] >= 7`; `result["FloodRiskCategory"] in ["High", "Very High"]`	Pass	0.000s
Far from thames lower risk	`result["FloodRiskScore"] <= 6`; `result["FloodRiskCategory"] in ["Low", "Medium"]`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Flood stages ordering	`alert < warning < severe`	Pass	0.001s
Processing stats	`stats["successful_gauges"] == 5; stats["failed_gauges"] == 0`	Pass	0.001s
Lines 264-266: exception during json.dump is logged and re-raised.	—	Pass	0.000s
Line 245: end_time is set by the first generate(); the second generate() call	—	Pass	0.001s
Line 290: non-dict section_schema returns {}.	`result == {}`	Pass	0.000s
Line 294: 'type', 'options', 'description', 'values' keys are skipped.	`skip not in result`	Pass	0.000s
Lines 159-162: count ¿ max_gauges → capped to max_gauges.	`len(result["data"]["flood_gauges"]) == max_available`	Pass	0.006s
Line 155-156: count ¡ 1 raises ValueError.	—	Pass	0.000s
Lines 354-355: generate_gauges() convenience function.	`"data" in result; len(result["data"]["flood_gauges"]) == 3`	Pass	0.001s
Lines 212-215: exception in _generate_single_gauge increments failed_gauges.	`result["processing_stats"]["failed_gauges"] == 1`	Pass	0.000s
Lines 122-125: log() dispatches to correct logger method.	—	Pass	0.000s
Lines 309-327: missing FloodGauge/Header/Location/FloodStages are created.	`gauge_data["FloodGauge"]["Header"]["GaugeID"] == "GAUGE-t...; gauge_data["FloodGauge"]["Location"]["GaugeLatitude"] == ...`	Pass	0.000s
Lines 99-100: verbose=False sets logger to WARNING.	`logging.getLogger("port.src.gauge.gauge").level == loggin...`	Pass	0.000s
Output file exists	`result["file_path"].exists()`	Pass	0.001s
Output file is valid json	`"flood_gauges" in data`	Pass	0.002s
Output json has correct count	`len(data["flood_gauges"]) == 5`	Pass	0.002s
Lines 378-381: __main__ block calls generate_gauges and logs results.	`(tmp_path / 'gauge.json').exists()`	Pass	0.005s
Gauge id format	`re.match(r"^GAUGE-[a-f0-9]{8}$", gid)`	Pass	0.001s
Gauge ids unique	`len(ids) == len(set(ids))`	Pass	0.001s
Generate correct count	`len(result["data"]["flood_gauges"]) == 5`	Pass	0.001s

Test Description	Acceptance Criteria	Result	Time
Generate returns expected keys	`"data" in result`; `"file_path" in result` (+1 more)	Pass	0.003s
Locations have coordinates	`"lat" in loc`; `"lon" in loc` (+1 more)	Pass	0.001s
Date field returns date string	`isinstance(val, str)`; `re.match(r"^\{}d{4}-\{}d{2}-\{}d{2}$", val)`	Pass	0.000s
Decimal field returns float	`isinstance(val, (int, float))`	Pass	0.000s
Gauge name includes area	`"TestArea" in val`	Pass	0.000s
Menu field returns valid option	`isinstance(val, str)`	Pass	0.000s
Text field returns string	`isinstance(val, str)`; `len(val) > 0`	Pass	0.000s
Catchment name stored	`meta["catchment_name"] == "Thames"`	Pass	0.000s
Flood stage ordering	`meta["flood_alert"] < meta["flood_warning"]`; `meta["flood_warning"] < meta["severe_flood_warning"]` (+1 more)	Pass	0.000s
Gauge id format	`re.match(r"^GAUGE-[a-f0-9]{8}$", meta["gauge_id"])`	Pass	0.000s
Historical high level range	`5.0 <= meta["historical_high_level"] <= 8.0`	Pass	0.000s
Install date format	`re.match(r"^\{}d{4}-\{}d{2}-\{}d{2}$", meta["install_date"])`	Pass	0.000s
Returns dict with required keys	`key in meta`	Pass	0.000s
Equatorial degree approx 111km	`110 < d < 113`	Pass	0.000s
London to paris approx 340km	`320 < d < 360`	Pass	0.000s
Same point is zero	`model.haversine_km(0.0, 0.0, 0.0, 0.0) == pytest.approx(0...`	Pass	0.000s
Symmetric	`d1 == pytest.approx(d2, rel=1e-6)`	Pass	0.000s
Monotone increasing	`all(levels[i] < levels[i + 1] for i in range(len(levels) ...`	Pass	0.000s
Negative intensity returns zero	`model.intensity_to_level(-10.0, gauge) == 0.0`	Pass	0.000s
Positive intensity positive level	`model.intensity_to_level(50.0, gauge) > 0.0`	Pass	0.000s
Sensitivity scales output	`model.intensity_to_level(50, g2) == pytest.approx( ...`	Pass	0.000s
Uses historical high when set	`lv_with != lv_without`	Pass	0.000s
Zero intensity returns zero	`model.intensity_to_level(0.0, gauge) == 0.0`	Pass	0.000s

1.1 Test Document History

Date	Version	Description	Author
05-Mar-2026	1.0	Initial test suite (26 tests)	D. Kelly
07-Mar-2026	1.1	23 test(s) added; 26 test(s) removed (total: 23)	D. Kelly
09-Mar-2026	1.2	40 test(s) added (total: 63)	D. Kelly
14-Mar-2026	1.3	14 test(s) added (total: 77)	D. Kelly
26-Mar-2026	1.4	4 test(s) added (total: 81)	D. Kelly